Unidad 5.1. Leaflet



Contenido de la unidad

- ✓ ¿Qué es Leaflet?
- ✓ El mapa
- ✓ Capas
- ✓ Controles
- ✓ Eventos
- ✓ Plugins



mappingGIS

¿Qué es Leaflet?

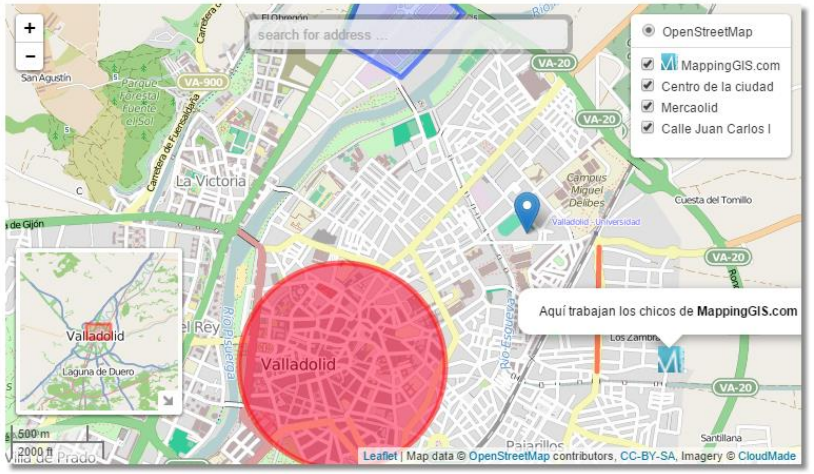


¿Qué es Leaflet?

Leaflet **es una librería JavaScript *open source*** para crear mapas interactivos en un entorno móvil.

Leaflet es la elección utilizada por Flickr, Wikimedia, foursquare, OSM, CARTO, GIS Cloud, Washington Post, Wall Street Journal, Geocaching.com, etc.

Surgió a principios de **2011**. Creado por Vladimir Agafonkin, ingeniero de Mapbox con un equipo de +8000 colaboradores especializados.
2012 - OpenStreetMap adopta Leaflet.
2013 - Aparecen Plugins clave como:leaflet.draw, leaflet.markercluster, leaflet.fullscreen...
2017–2020 - Integración con frameworks: React, Vue, Angular. Uso en apps web de QGIS (como qgis2web).
2024–2025 - La comunidad mantiene vivo el proyecto. Plugins modernos, integraciones con Svelte, React y compatibilidad con OGC APIs



mappingGIS

¿Qué es Leaflet?

Leaflet = Sencillez

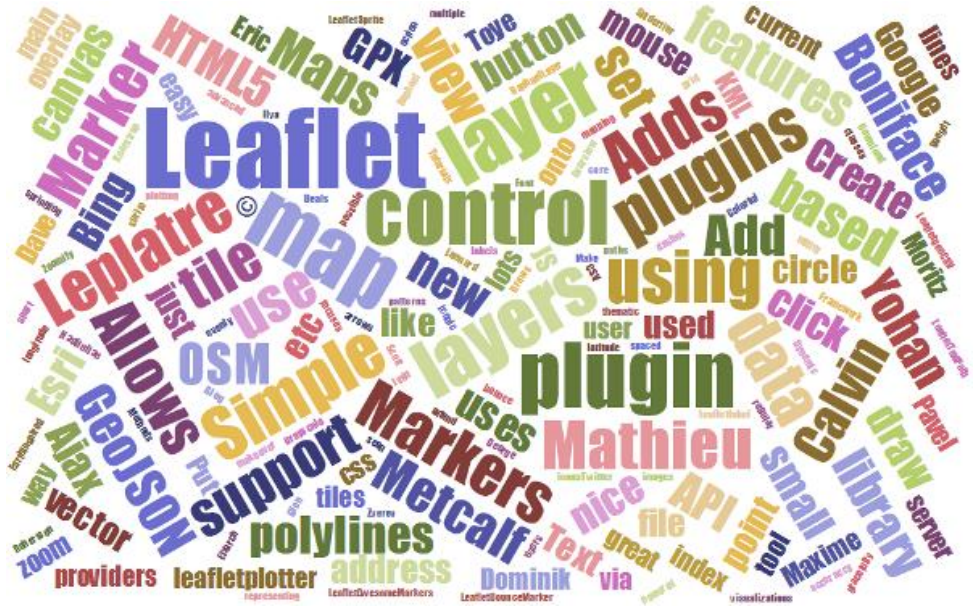


Imagen propiedad de Malcolm Maclean. Leaflet Tips and Tricks

- Muy ligero (39KB)
- Sencillo y rápido de aprender
- API bien documentada y fácil de usar
- Características básicas pero que funcionan a la perfección
- Soporte móvil
- Funciona tanto en los modernos como en los viejos navegadores web
- Ampliable con plugins para extender su funcionalidad

mappingGIS

Documentación

Documentación de la API:
http://leafletjs.com/reference.html

Ejemplos y tutoriales:
http://leafletjs.com/examples.html

Código:
https://github.com/Leaflet/Leaflet

Leaflet Tips and Tricks
https://leanpub.com/leaflet-tips-and-tricks/read

mappingGIS

Comenzando con Leaflet. Carga de la librería

El encabezado del documento carga necesariamente los archivos JavaScript y CSS que serán incluidos con el mapa:

- La librería JavaScript **leaflet.js**
- La hoja de estilo **leaflet.css**

```
<link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css" integrity="sha256-p4NxAoJBhIIN+hmNHrzRCf9tD/miZyoHS5obTRR9BMY=" crossorigin=""/>

<script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js" integrity="sha256-20nQCchB9co0qIjJZRGuk2/Z9VM+kNiyxNV1lvTlZBo=" crossorigin=""></script>
```

mappingGIS

Comenzando con Leaflet. Carga de la librería

Leaflet se puede utilizar **descargando la librería** o mediante un enlace **CDN**.

Descarga

https://leafletjs.com/download.html

```
<link rel="stylesheet" href="/path/to/leaflet.css" />
<script src="/path/to/leaflet.js"></script>
```

Version	Description
Leaflet 1.9.4	Stable version, released on May 18, 2023.
Leaflet 1.8.0	Previous stable version, released on April 18, 2022.
Leaflet 2.0-dev	In-progress version, developed on the `main` branch.

Enlace CDN

```
<link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css" />
<script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js" ></script>
```

mappingGIS

El mapa- L.map()



El mapa- L.map()

L.map → crear y manipular el mapa.

Activadas por defecto

- Interacciones del ratón sobre el mapa.
- Control de zoom y atribución.

Sintaxis

L.map(<HTMLElement|String> id,
<Map options> opciones)

L.map Crea una instancia a un objeto mapa definido por un elemento div .

```
var map = new L.map('map', {
        center: ...
        zoom: ..
        layers :...
});
```

→ Opciones de la clase map

El mapa- L.map()

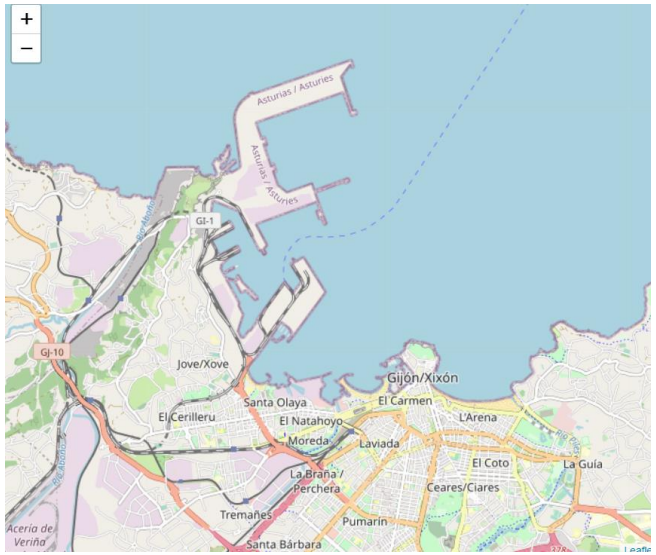
Creación de un ** ** → espacio en la página para albergar el mapa.

div

```
<div id = "map"> </div>
```

Estilo

```
<style>
        #map {
        width: 700px;
         height: 600px; }
</style>
```



mappingGIS

Mi primer mapa - L.map()

Un mapa básico= L.map + L.tileLayer

```
var map = L.map('map', {
        center: [40.965, -5.664],
        zoom: 16
        });
L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png').addTo(map);
```



mappingGIS

Opciones de la clase L.map()

Principales

- **center** es el centro geográfico inicial del mapa, definido por sus coordenadas geográficas.
- **zoom:** zoom inicial del mapa.
- **layers** son las capas que se añadirán al mapa inicialmente.

Secundarias

- **minZoom** Mínimo nivel de zoom
- **maxZoom** Máximo nivel de zoom
- **maxBounds** Cuando se establece esta opción, el mapa restringe la vista a las coordenadas geográficas establecidas, llevando al usuario de nuevo a las coordenadas cuando intente desplazarse fuera de dichas coordenadas.
- **crs** Sistema de referencia de coordenadas. Por defecto es EPSG3857 (Proyección WGS84)

mappingGIS

Métodos para modificar el estado del mapa - L.map

setView( ) Define la **vista** del mapa (con un centro geográfico y un zoom).

Sintaxis

```
setView( <LatLng> center,
         <Number> zoom?,
         <zoom/pan options> options? )
```

Ejemplo

```
var map = L.map('map').setView([-41.2858, 174.78682], 14);
```

Otros métodos para modificar el estado del mapa:

`setZoom(), zoomIn(), zoomOut(), setZoomAround(), fitBounds(), fitWorld(), panTo(), panInsideBounds(), panBy(), invalidateSize(), invalidateSize(), setMaxBounds(), locate(), stopLocate(), remove(), flyTo()`

mappingGIS

Métodos para recuperar el estado del mapa - L.map

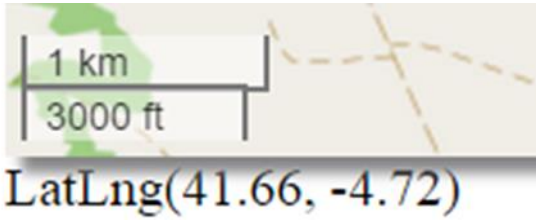
Métodos Get

getCenter( ) Retorna las coordenadas del centro geográfico de la vista del mapa.

Ejemplo

```
var center = map.getCenter();

document.write(center);
```



getZoom() → Retorna el **nivel de zoom** de la vista actual.

getMinZoom() →Retorna el **mínimo nivel de zoom** del mapa.

getMaxZoom() → Retorna el **máximo nivel de zoom** del mapa.

getBounds() → Retorna **LatlngBound** de la vista actual del mapa (las coordenadas de las esquinas suroeste y noreste del mapa)

Otros métodos :getBoundsZoom(), getSize(), getPixelBounds(), getPixelOrigin()

mappingGIS

Métodos para trabajar con capas y controles

addLayer( ) ⟷ **removeLayer( )**

Añadir una capa al mapa.

```
addLayer( <ILayer> layer,
    <Boolean> insertAtTheBottom? )
```

Eliminar una capa del mapa.

Trabajar con capas

Primero instanciamos una capa de Google:

```
    var ggl = L.gridLayer.googleMutant({
        type: 'roadmap'
      });
```

Y la añadimos al mapa:

```
    map.addLayer(ggl);
```

Ejemplo

Otros métodos que sirven para trabajar con capas y controles:

`hasLayer, eachLayer, openPopup, closePopup, addControl, removeControl`

mappingGIS

Capas



Las capas -Ilayer

Leaflet → 4 grupos para representar capas.

Capas raster

- TileLayer
- TileLayer.WMS
- VideoOverlay
- ImageOverlay

Capas vectoriales

- Path
- Polyline
- Polygon
- Rectangle
- Circle
- CircleMarker
- SVGOverlay
- SVG
- Canvas

Capas de interfaz de usuario

- Markers
- Popups
- Tooltip

Otras capas

- LayerGroup
- FeatureGroup
- GeoJSON
- GridLayer

mappingGIS

Capas raster

Leaflet → 4 clases para capas raster.

Capas raster

- Capas teseladas – tileLayer
- Capas teseladas WMS – tileLayer.WMS
- Capas de imágenes – imageOverlay
- Capas de vídeo - VideoOverlay

mappingGIS

Capas teseladas – L.tileLayer

L.tileLayer → cargar y visualizar capas teseladas en el mapa

Su sintaxis es:

```
L.tileLayer( <String> urlTemplate,
             <TileLayer options> opciones? )
```

La **urlTemplate** es una cadena de texto con el formato:

```
http://{s}.mapdomain.org/{z}/{x}/{y}.png,
```

{s} → permitirá a uno de los subdominios del dominio principal que sea utilizado. Estos se utilizan normalmente en secuencia para cargar el mapa más rápido al permitir múltiples solicitudes paralelas.
{z} → declara el nivel de **zoom**.
{x} e **{y}** →definen las **coordenadas** de las teselas.

Ejemplo

```
L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png').addTo(map);
```

mappingGIS

Opciones de capas teseladas – L.tileLayer

minZoom, maxZoom, tileSize, attribution, detectRetina, opacity, bounds...

La atribución →es la colocación de una referencia adecuada para la fuente de los datos. La idea es asegurar que los créditos (y el reconocimiento de derechos de autor) se proporcionan al proveedor de teselas como es razonable.

Ejemplo

```
var OpenStreetMap_Mapnik = L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
maxZoom: 19,
attribution: '&copy; <a href="https://www.openstreetmap.org/copyright">OpenStreetMap</a> contributors'
});
```

mappingGIS

Métodos de capas teseladas – L.tileLayer

addTo , bringToFront, bringToBack, setOpacity, setZIndex, redraw, setUrl, getContainer

addTo → para añadir la capa directamente al mapa.

Ejemplo

```
L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png').addTo(map);
```

mappingGIS

Capas teseladas WMS - L.tileLayer.WMS

L.tileLayer.WMS → se utiliza para visualizar **servicios WMS** como capa de teselas en el mapa.
Extiende la clase **L.TileLayer**

Sintaxis:

L.tileLayer.wms(<String> baseUrl,
<TileLayer.WMS options> opciones)

Ejemplo

```
var cities = L.tileLayer.wms("http://localhost:8080/geoserver/wms", {
 layers: 'cities', // obligatorio
 format: 'image/png',
 transparent: true,
 attribution: "Natural Earth",
 styles: "cities_label"
});
```

mappingGIS

Opciones de las capas teseladas WMS - L.tileLayer.WMS

La clase **L.tileLayer.WMS** incluye todas las opciones de la clase **L.TileLayer** y además:

- **layers** →**(requerido)** Lista de capas WMS (separada por comas) para mostrar.
- **styles** →lista de estilos WMS.
- **format** →formato de las imágenes WMS (usa 'image / png' para capas con transparencia).
- **transparent** → Si es true, el servicio WMS devolverá las imágenes con transparencia. Por defecto está establecido como false
- **version**. →Versión del servicio WMS de usar. Por defecto 1.1.1.
- **crs** →Sistema de coordenadas de referencia que se utilizará para las peticiones WMS, por defecto el CRS del mapa.

Ejemplo

```
var cities = L.tileLayer.wms("http://localhost:8080/geoserver/wms", {
 layers: 'cities', // obligatorio
 format: 'image/png',
 transparent: true,
 attribution: "Natural Earth",
 styles: "cities_label"
});
```

mappingGIS

Capas de imágenes - L.imageOverlay

L.imageOverlay → añade imagen georeferenciada al mapa.

Sintaxis:

```
L.ImageOverlay(
        <String> imageUrl,
        <LatLngBounds> bounds,
        <ImageOverlay options> opciones? )
```

Opciones: opacity, attribution...
Métodos: addTo, bringToFront, bringToBack, setOpacity, setUrl

Requisito: Georeferenciar imagen



Ejemplo

```
var imageUrl = 'http://www.lib.utexas.edu/maps/historical/newark_nj_1922.jpg',
    imageBounds = [[40.712216, -74.22655], [40.773941, -74.12544]];

L.imageOverlay(imageUrl, imageBounds).addTo(map);
```

mappingGIS

Capas de interfaz de usuario

Capas de interfaz de usuario

- Marcadores - Markers
- Ventanas emergentes -Popups
- Etiquetas - Tooltip

mappingGIS

Marcadores - L.marker

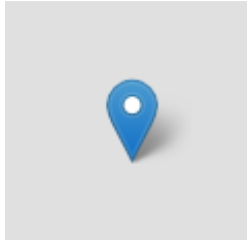
L.marker → muestra un marcador/icono sobre el mapa.

Sintaxis:

```
L.marker( <LatLng> latlng,
          <Marker options> opciones? )
```

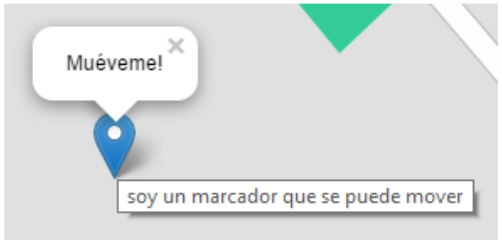
Ejemplo

```
L.marker([41.66, -4.71])
.addTo(map)
```



Opciones de los marcadores - L.marker

•**icon** → **L.Icon** se utilizar para representar el marcador y seleccionar el icono.
•**clickable** →Por defecto **true**. Si es falso, el marcador no emitirá eventos de ratón.
•**Draggable**→ Por defecto **false**. Si es true, es posible arrastrar el marcador con el movimiento del ratón o con un toque.
•**Keyboard** → Por defecto **true**. Permite que el marcador sea controlado por el tabulador del teclado y hacer clic sobre el pulsando intro.
•**Title** → Texto que se muestra sobre el marcador (tooltip) al pasar el ratón por encima de el. Por defecto no hay tooltip.
•**alt** →Texto para el atributo alt de la imagen del icono (**útil para accesibilidad**).
•**zIndexOffset** →pone el marcador en la parte superior de todos los demás (o por debajo)
•**opacity** → opacidad del marcador.
•**riseOnHover** →Si es true, el marcador se pondrá en la parte superior de los demás cuando se pase el ratón sobre él.
•**riseOffset** →El desplazamiento z-index utilizado para la opción riseOnHover.

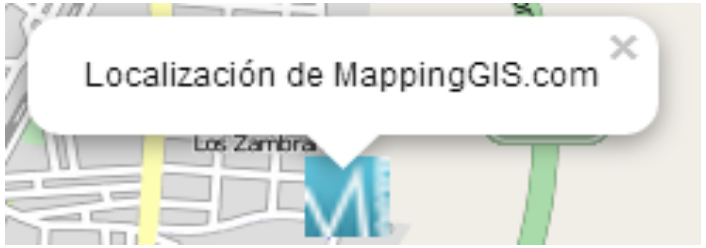


mappingGIS

Métodos de los marcadores - L.marker

addTo, getLatLng, setLatLng, setIcon, setZIndexOffset, setOpacity, bindPopup, unbindPopup, openPopup, getPopup, closePopup, togglePopup, setPopupContent, toGeoJSON

Destacar **addTo** para añadir directamente al mapa y **bindPopup**, que lanza una ventana emergente con contenido HTML al hacer clic sobre el marcador. También podemos abrir un popup con el método openPopup



Ejemplo

```
L.marker([41.65246, -4.6982], {icon: myIcon})
.addTo(map)
.bindPopup("Localización de <b>MappingGIS.com </b>");
```

mappingGIS

Ventanas emergentes - L.popup

L.popup → abrir ventanas emergentes en ciertos lugares del mapa.

La sintaxis es la siguiente:

```
L.popup( <Popup options> options?, <ILayer> source? )
```

Ejemplo

```
var popup = L.popup()
    .setLatLng([41.65, -4.70])
    .setContent('<p>Hola!<br/>Soy un bonito popup.</p>')
    .openOn(map);
```

Esta es una forma más complicada que las que hemos visto, ya que para mostrar popups en marcadores es más fácil utilizar el método bindPopup



mappingGIS

Etiquetas - L.tooltip ()

L.tooltip → se usa para visualizar pequeños textos sobre capas.

La sintaxis es la siguiente:

```
L.tooltip(<Tooltip options> options?, <Layer> source?)
```

Ejemplo

```
var marker = L.marker([41.66, -4.71])
 .addTo(map)

marker.bindTooltip("León - España",
    {permanent:true}
).openTooltip();
```



Opciones de etiquetado - L.tooltip ()

- **pane -** Tipo de panel (elemento HTML) en el que se agregará la etiqueta.
- **offset -** Desplazamiento de la posición de la etiqueta. (por defecto **Point (0,0)**).
- **direction -** Dirección hacia la que se abrirá la etiqueta. Los valores posibles son: right, left, top, bottom, center, auto. En modo auto cambiará dinámicamente entre derecha e izquierda de acuerdo a la posición de las etiquetas en el mapa (por defecto **'auto').**
- **permanent** - Si deseamos abrir la etiqueta de forma permanente o sólo al pasar el ratón por encima del marcador. Por defecto el valor es **false.**
- **sticky** - Si es verdadero, la etiqueta seguirá el ratón en lugar de estar fija en el centro del objeto geográfico. Por defecto el valor es **false**.
- **interactive** - Si es cierto, la información de herramienta va a escuchar a los eventos de características. Por defecto el valor es **false.**
- **opacity** - Opacidad del contenedor de la etiqueta. Por defecto el valor es **0.9.**

mappingGIS

Capas vectoriales

Capas vectoriales

- Path
- Polyline
- Polygon
- Rectangle
- Circle
- CircleMarker
- SVGOverlay
- SVG
- Canvas

mappingGIS

Capas Vectoriales -Path

Path es una clase abstracta que contiene opciones y constantes compartidas por las capas vectoriales: *Polygon, Polyline, Circle*

No se usa directamente.
Las **opciones** de esta clase son:

Stroke, color, weight, opacity, fill, fillcolor, fillOpacity, dashArray, lineCap, lineJoin, clickable, pointerEvents, classname

Los **eventos:**
Click, dblclick, mousedown, mouseover, mouseout, contextmenu, add, remove, popupopen, popupclose

Métodos:
addTo, bindPopup, unbindPopup, openPopup, closePopup, setStyle, getBounds, bringToFront, bringToBack, redraw

mappingGIS

Capas Vectoriales -Polyline

L.Polyline → Línea.
Extiende la clase Path.
Para crear una polilínea debemos especificar las coordenadas de los vértices.

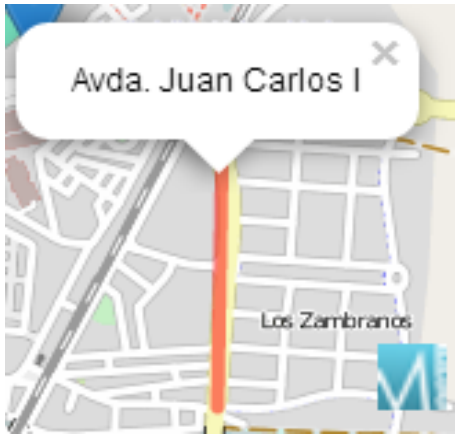
La sintaxis es la siguiente:

L.polyline(<LatLng[ ]> latlngs,
<Polyline options> options?)

Ejemplo:

```
var pointA = new L.latLng(41.65925, -4.70404);
var pointB = new L.latLng(41.65166, -4.70421);
var latlngs = [pointA, pointB];

var polyline = L.polyline(  latlngs,
{color: 'blue'}
).addTo(map).bindPopup ('Avda. Juan Carlos I');
```



mappingGIS

Capas Vectoriales -Polygon

L.polygon→ Polígono.
Extiende la clase Polyline.

Para crear un polígono debemos especificar las coordenadas de los vértices.

La sintaxis es la siguiente:

L.polygon(<LatLng> latlngs,
<Polyline options> options?)

Ejemplo:

```
var polygon = L.polygon([
[41.67778, -4.72163],
[41.67349, -4.71511],
[41.66958, -4.72017],
[41.67169, -4.72515]
])
.addTo(map)
.bindPopup ('Mercaolid');
```



mappingGIS

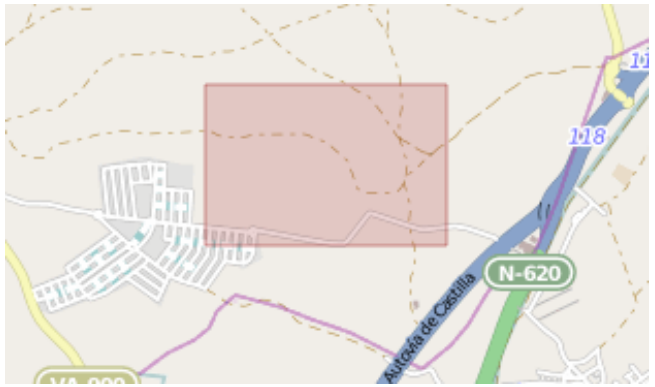
Capas Vectoriales -Rectangle

L.Rectangle → Rectángulo.
Extiende la clase Polygon.

Para crear un rectángulo debemos especificar los límites geográficos.

La sintaxis es la siguiente:

L.rectangle(<LatLngBounds> bounds,
<Path options> options?)



Ejemplo:

```
var bounds = [[41.71, -4.72], [41.70, -4.74]];

L.rectangle(bounds,
{color: "brown",
weight: 1}
).addTo(map);
```

mappingGIS

Capas Vectoriales -Circle

L.circle → círculo.
Extiende la clase Path.

La sintaxis es la siguiente:

```
L.circle( <LatLng> coordenadas del centro,
          <Number> radio,
          <Path options> opciones )
```

Ejemplo:

```
var circle = L.circle([41.658, -4.725],
700, {
    color: 'red',
    fillColor: '#f03',
    fillOpacity: 0.5
}
).addTo(map);
```



mappingGIS

Capas Vectoriales -CircleMarker

L.circleMarker →Circunferencias.
Extiende la clase Circle.
La sintaxis es la siguiente:

L.circleMarker(<LatLng> latlng,
<Path options> options?)

Ejemplo:

```
var circleMarker = L.circleMarker([41.63, -4.775], {
    radius: 20
})
.addTo(map);
```

El tamaño del radio viene especificado en pixels. Por lo tanto al modificar el zoom del mapa, el tamaño de la circunferencia mantiene siempre el mismo tamaño.



mappingGIS

Capas Vectoriales – SVG y Canvas

Tanto SVG (Gráficos vectoriales escalables) como Canvas son métodos de renderización del mapa.

Ambos heredan las propiedades de la clase base Renderer.

Permite que las capas vectoriales se visualicen utilizando alguno de estos métodos. Debido a limitaciones técnicas, SVG no está disponible en todos los navegadores web, especialmente con IE 7 y IE8

Ejemplo

```
var map = L.map('map', {
    renderer: L.svg();
});
```

```
var map = L.map('map', {
            renderer: L.canvas();
});
```

mappingGIS

Otras capas

Otras capas

- LayerGroup
- FeatureGroup
- GeoJSON
- GridLayer

mappingGIS

Grupo de capas – L.layerGroup()

L.layerGroup → se utiliza para agrupar varias capas y manejarlas como una sola.
Extiende la clase **Layer.**

Si agregamos un grupo al mapa, cualquier capa que añadamos o eliminemos del grupo, también se añadirá o eliminará del mapa.

Sintaxis:

```
L.layerGroup( <ILayer[]> layers? )
```

Ejemplo

```
var littleton = L.marker([39.61, -105.02]).bindPopup('Littleton, CO.'),
    denver    = L.marker([39.74, -104.99]).bindPopup('Denver, CO.'),
    aurora    = L.marker([39.73, -104.8]).bindPopup('Aurora, CO.'),
    golden    = L.marker([39.77, -105.23]).bindPopup('Golden, CO.');


var cities = L.layerGroup([littleton, denver, aurora, golden]);
```

mappingGIS

Grupo de objetos geográficos – L.featureGroup()

L.featureGroup extiende la clase LayerGroup, pero la primera también tiene eventos de ratón y comparte el método bindPopup.

Sintaxis:

```
L.featureGroup( <ILayer[]> layers? )
```

Ejemplo

```
L.featureGroup([marker1, marker2, polyline])
    .bindPopup('Hello world!')
    .on('click', function() { alert('Clicked on a group!'); })
    .addTo(map);
```

GeoJSON

GeoJSON → representa un objeto GeoJSON o un listado de objetos GeoJSON. Extiende la clase FeatureGroup.

Esta clase nos permite utilizar datos GeoJSON y visualizarlos en el mapa.

Sintaxis:

```
L.geoJson( <Object> geojson?, <GeoJSON options> options? )
```

Ejemplo

```
L.geoJSON(data, {
    style: function (feature) {
        return {color: feature.properties.color};
    }
}).bindPopup(function (layer) {
    return layer.feature.properties.description;
}).addTo(map);
```

Opciones de L.geoJSON()

- **pointToLayer** - Función que se utiliza para la creación de capas de puntos de GeoJSON (si no se especifica, se crearán marcadores simples).
- **style** - Función que se utiliza para obtener los estilos aplicados a las capas vectoriales de tipo línea y polígono.
- **onEachFeature** - Se utiliza para llamar a eventos y desplegar pop up de los *features* de GeoJSON.
- **Filter** - Esta función se utiliza para decidir si un objeto geográfico (*feature*) es mostrado en el mapa o no.
- **coordsToLatLng** - Función que se utiliza para convertir las coordenadas GeoJSON en puntos LatLong.

mappingGIS

Otras capas - GridLayer

Clase genérica para el manejo de una malla teselada de elementos HTML. Esta es la clase base para todas las capas teseladas y sustituye a la antigua clase TileLayer.Canvas.

GridLayer se puede extender para crear una malla teselada de elementos HTML, como `<canvas>`, `<img>` o `<div>`. GridLayer se encarga de crear y animar estos elementos DOM por nosotros.

Sintaxis:

```
L.gridLayer(<GridLayer options> options?)
```

Ejemplo

```
var CanvasLayer = L.GridLayer.extend({
    createTile: function(coords){
        // create a <canvas> element for drawing
        var tile = L.DomUtil.create('canvas', 'leaflet-tile');
        // setup tile width and height according to the options
        var size = this.getTileSize();
        tile.width = size.x;
        tile.height = size.y;
        // get a canvas context and draw something on it using coords.x,
coords.y and coords.z
        var ctx = tile.getContext('2d');
        // return the tile so it can be rendered on screen
        return tile;
    }
});
```

mappingGIS

Controles



Controles

Los controles permiten a los usuarios navegar con movimientos del ratón (por ejemplo, arrastrar para moverse, doble clic para ampliar y los botones de acercar y alejar.

Por defecto solo están habilitados los controles de **zoom** y **atribución**.

Los 4 controles de Leaflet son:

- **Zoom**
- **Attribution**
- **Layers**
- **Scale**

mappingGIS

Controles – L.control()

Añadir control→ **L.control**. Implementa la interfaz IControl.

Podemos agregar controles al mapa de la siguiente manera:

control.addTo(map); **map.addControl(control);**

Sintaxis

```
L.control( <Control
options> opciones )
```

Opciones

position: la posición inicial del control en el mapa.
Las posiciones posibles son: **topleft, topright, bottomleft, bottomright**.
Por defecto es topright

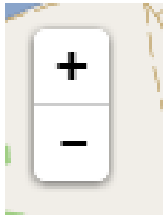
Métodos

setPosition, getPosition, addTo, removeFrom, getContainer

mappingGIS

Controles – L.control.zoom

La clase **L.control.zoom** sirve para añadir un control de zoom básico.



Incluye dos botones (acercar y alejar la imagen).

Se añade al mapa de forma predeterminada (a menos que establezca la opción ZoomControl a false).

Extiende la clase L.control.

```
L.control.zoom( <Control.Zoom options> opciones )
```

Su posición por defecto es topleft.

Controles – L.control.attribution

El control de atribución permite visualizar los datos de atribución en un pequeño cuadro de texto dentro del mapa.

Se añade al mapa por defecto, a menos que establezcamos la opción del mapa ***attributionControl*** a **false.** Extiende la clase L.control.

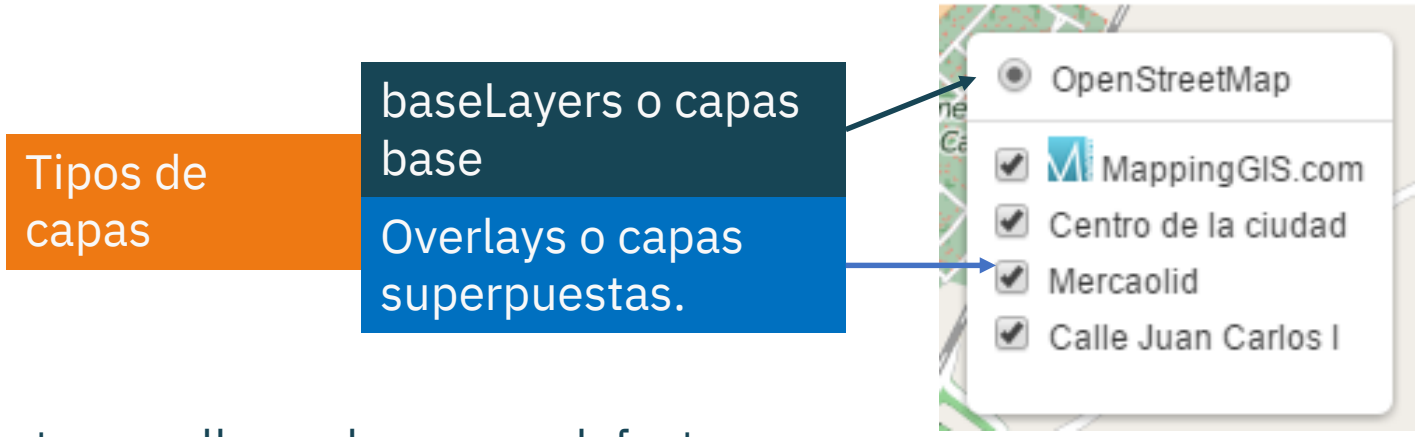
Su posición por defecto es bottomright.



Controles – L.control.layers

L.control.layers → crea un control de capas.
El control de capas da al usuario la posibilidad de alternar la visibilidad de las capas, mostrándolas o escondiéndolas.

Extiende la clase L.control. Su posición por defecto es topright.



Una opción interesante es collapsed, que por defecto es true, si se establece a false se verá el control expandido.

```
L.control.layers(baseLayers, overlays,{collapsed:false}).addTo(map);
```

mappingGIS

Controles – L.control.scale

Es un sencillo control que muestra la escala del centro actual de la pantalla en el sistema métrico (m/km) e imperial (millas/pies). Extiende la clase L.control.

Sintaxis:

L.control.scale(<Control.Scale options> options?)

Su posición por defecto es bottomleft.

Las opciones metric e imperial vienen habilitadas por defecto, pero se pueden cambiar a false:

```
L.control.scale({imperial: false}).addTo(map);
```



mappingGIS

Eventos



Métodos de evento

Cada vez que algo ocurre en Leaflet, por ejemplo, cuando el usuario hace clic en un marcador o el mapa cambia de zoom, el objeto correspondiente envía un evento que se puede suscribir a una función. Esto nos permite reaccionar a la interacción del usuario.

Ejemplo:

```
map.on('click', function(e) {
      alert(e.latlng);
      });
```

Leaflet trabaja con detectores de eventos de referencia, por lo que si queremos agregar un oyente y luego quitarlo, lo debemos definir como una **función:**

```
function onClick(e) { ... }

map.on('click', onClick);
map.off('click', onClick);
```

mappingGIS

Objetos de evento

Un objeto de evento es un objeto que recibimos como argumento en una función de escucha cuando algún evento se activa, conteniendo información útil sobre ese evento.

En el ejemplo:

```
map.on('click', function(e) {
  alert(e.latlng);
  });
```

Click es el tipo de evento.
e es un objeto de evento.
latlng es el punto geográfico donde ocurre el evento del ratón

Plugins



Plugins

Ya sabemos que Leaflet es una librería JavaScript *open source* que se centra en un conjunto básico de características, de esto modo pretende mantenerse lo más ligera posible.

Esta plataforma funciona bien para el usuario normal cubriendo sus necesidades básicas, pero si queremos más funcionalidad debemos recurrir al uso de complementos o **plugins**.

Uno de los aspectos clave de Leaflet es que ofrece un buen código base sobre el que se pueden desarrollar complementos para hacer una aplicación más potente.

Te recomendamos visitar nuestro blog para ver:

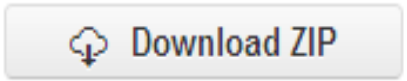
Los mejores plugins para Leaflet

mappingGIS

¿Cómo incorporo los plugins a mi mapa de Leaflet?

1. Accedemos al repositorio oficial de plugins:

2. Nos descargamos el plugin haciendo clic sobre el botón Download ZIP y descomprimimos el archivo.



3. Añadimos la librería en la cabecera del HTML. Por ejemplo:

```
<script src=”leaflet-master\layer\tile\ejemplo.js”></script>
```

4. Ya podemos utilizar la nueva librería. Por ejemplo:

```
L.control.mousePosition().addTo(map);
```

mappingGIS



Ahora puedes hacer los ejercicios de esta unidad

mappingGIS